4 Simple square demonstration

We now apply the construction of Section 2 and the protocol of Section 3 to the simplest controlled case: a square plate pulled uniformly at one end, whose expected stretch pL/E lets us check every computed number against physics.

The plate occupies $0 \le x, y \le L$ with $L = 1$ m and has thickness $t = 0.01$ m, $E = 200$ GPa and $\nu = 0.3$, in plane stress without body forces (Fig. 1a). The left edge is clamped, $u_x = u_y = 0$. The right edge carries the uniform traction $\bar{\tau} = (p, 0)$ with $p = 100$ MPa, so $q = tp = 10^6$ N/m with resultant $ptL = 10^6$ N; the top and bottom edges are free; the load is a prescribed traction, not a prescribed displacement. Units are m, Pa and N. The mesh has $m \times m$ square cells, each cut along its bottom-left to top-right diagonal, giving $(m+1)^2$ nodes and $2m^2$ triangles. The clamp prescribes $2(m+1)$ components, leaving $N = 2m(m+1)$ unknowns. The case is `force_driven` in `fem/plate.py`; all numbers below come from `draft/experiments/square.py`.

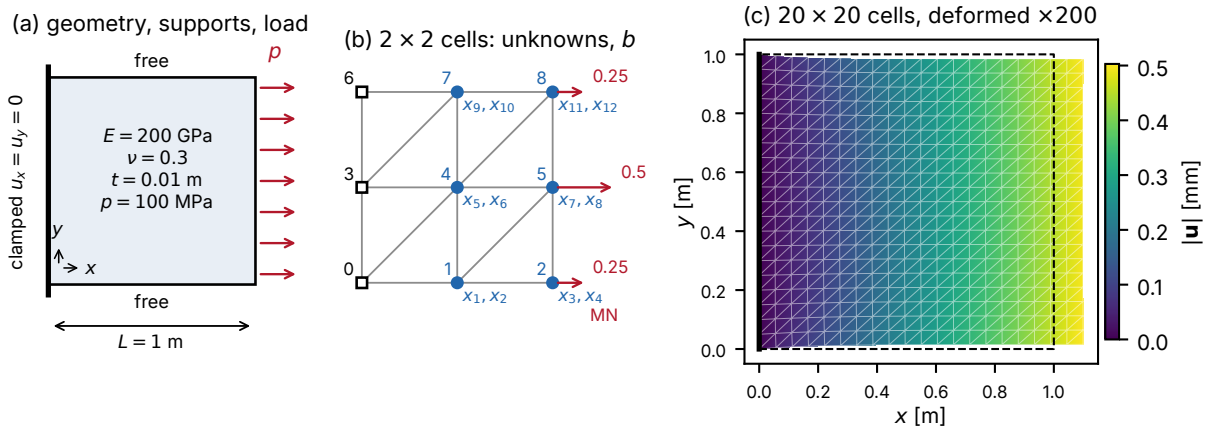


Figure 1: (a) Geometry, supports and load. (b) The 2×2-cell mesh: clamped nodes (squares), free nodes (dots) with their unknowns $x_1, \ldots, x_{12}$, and the nodal forces forming b. (c) Original outline (dashed) and deformed 20×20-cell plate, displacements amplified by 200; colour is the actual displacement magnitude in mm.

4.1 A small system solved directly

For $m = 2$ there are 9 nodes, 8 triangles and 18 displacement components. Clamping nodes 0, 3 and 6 leaves $N = 12$ unknowns, ordered node by node as $x = (u_{x,1}, u_{y,1}, u_{x,2}, \ldots, u_{y,8})^T$ (Fig. 1b). Since $d_C = 0$, $b = f_F$. Each loaded edge of length 0.5 m carries 0.5 MN, split equally between its end nodes, so $b = 2.5 \times 10^5\,(0,0,1,0,0,0,2,0,0,0,1,0)^T$ N. For $\nu = 0.3$ every entry of A is an integer multiple of $Et/(40(1-\nu^2)) = 5.495 \times 10^7$ N/m:

$$A = \frac{Et}{40(1-\nu^2)} \begin{bmatrix} 54 & -13 & -20 & 6 & -14 & 13 & \cdot & -13 & \cdot & \cdot & \cdot & \cdot \\ -13 & 54 & 7 & -7 & 13 & -40 & -13 & \cdot & \cdot & \cdot & \cdot & \cdot \\ -20 & 7 & 27 & -13 & \cdot & \cdot & -7 & 6 & \cdot & \cdot & \cdot & \cdot \\ 6 & -7 & -13 & 27 & \cdot & \cdot & 7 & -20 & \cdot & \cdot & \cdot & \cdot \\ -14 & 13 & \cdot & \cdot & 108 & -26 & -40 & 13 & -14 & 13 & \cdot & -13 \\ 13 & -40 & \cdot & \cdot & -26 & 108 & 13 & -14 & 13 & -40 & -13 & \cdot \\ \cdot & -13 & -7 & 7 & -40 & 13 & 54 & -13 & \cdot & \cdot & -7 & 6 \\ -13 & \cdot & 6 & -20 & 13 & -14 & -13 & 54 & \cdot & \cdot & 7 & -20 \\ \cdot & \cdot & \cdot & \cdot & -14 & 13 & \cdot & \cdot & 54 & -13 & -20 & 7 \\ \cdot & \cdot & \cdot & \cdot & 13 & -40 & \cdot & \cdot & -13 & 54 & 6 & -7 \\ \cdot & \cdot & \cdot & \cdot & \cdot & -13 & -7 & 7 & -20 & 6 & 27 & \cdot \\ \cdot & \cdot & \cdot & \cdot & -13 & \cdot & 6 & -20 & 7 & -7 & \cdot & 27 \end{bmatrix}. \tag{4.1}$$

Dots are zeros, mostly between nodes without a common triangle, such as 1 and 7. A is exactly symmetric, with eigenvalues from $\lambda_{\min} = 1.073 \times 10^8$ to $\lambda_{\max} = 1.028 \times 10^{10}$ N/m, so

$\kappa_2(A) = 95.80$; the arrays are stored as CSV files in `draft/experiments/results/square/`.

Pivoted $PA = LU$, the explicit inverse and Cholesky $A = LL^T$ give relative residuals η (Eq. (3.4)) of 6.2×10^{-16}, 8.4×10^{-16} and 9.2×10^{-16}, and their solutions agree to 6.3×10^{-16} relative to their norm. From bottom to top, the right-edge nodes move $u_x = 0.5112$, 0.4822 and 0.4678 mm, close to $pL/E = 0.5$ mm, and $u_y = +0.1045$, $+0.0294$ and -0.0461 mm. The continuum problem is symmetric about $y = L/2$, which would make u_y antisymmetric; the deviation comes from the coarse mesh and its single diagonal direction.

4.2 A larger system and the iterative methods

For $m = 20$ the mesh has 441 nodes, 800 triangles and $N = 840$ unknowns. The sparse matrix stores 11,112 entries (9,590 numerically nonzero) and is symmetric up to rounding. Sparse estimates give $\lambda_{\min} = 1.917 \times 10^6$ and $\lambda_{\max} = 1.745 \times 10^{10}$ N/m, so $\kappa_2(A) = 9104$, in agreement with a dense computation to 10^{-14}. The scaled matrix S has eigenvalues in $[3.687 \times 10^{-4}, 2.960]$, so $\rho_J = 1.960$ by Eq. (3.2). Table 1 and Fig. 2 report the methods.

Table 1: All methods on the 840-unknown system under the protocol of Section 3 (best of three times; Linux x86-64, Python 3.13, SciPy 1.18.1). An asterisk marks failure to reach $\eta < 10^{-8}$.

Method	Iterations	Final η	Error vs LU	Time (ms)	Stop
Sparse LU (`splu`)	–	3.5×10^{-14}	–	$2.20 + 0.09$	factor + solve
Jacobi*	49	1.3×10^{10}	3.4×10^{7}	0.65	divergence guard
Damped Jacobi ($\omega = 0.5$)*	20,000	4.8×10^{-6}	8.7×10^{-5}	284	iteration cap
Gauss–Seidel	19,136	1.0×10^{-8}	1.7×10^{-7}	7,178	target
Steepest descent*	20,000	3.1×10^{-6}	4.2×10^{-5}	541	iteration cap
CG	242	8.8×10^{-9}	9.6×10^{-10}	5.4	target
Diagonal PCG	226	8.8×10^{-9}	4.9×10^{-10}	6.8	target
MINRES	236	9.4×10^{-9}	5.9×10^{-9}	13.7	target
Full GMRES (restart = N)	228	9.5×10^{-9}	6.0×10^{-9}	186	target

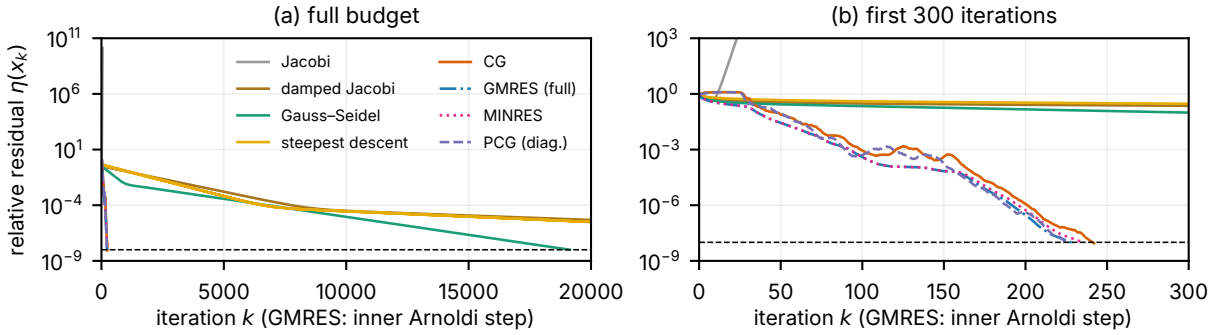


Figure 2: Relative residual $\eta(x_k)$ of every method on the 840-unknown system; the dashed line is the target 10^{-8}. (a) Whole budget. (b) First 300 iterations. GMRES shows its Arnoldi residual estimate; other curves are recomputed residuals.

The stationary methods behave as their diagnostics predict. Since $\lambda_{\max}(S) > 2$, Jacobi multiplies one error component by about -1.96 per sweep and is stopped by the guard after 49 sweeps. Damping with $\omega = 0.5$, below the limit $2/\lambda_{\max}(S) = 0.676$, restores convergence, but the slowest component shrinks only by $1 - \omega\lambda_{\min}(S) = 0.99982$ per sweep, a factor of 0.025 over the whole budget. Steepest descent is limited similarly: at most $(\kappa - 1)/(\kappa + 1)$ per step gives about 0.012 over 20,000 steps. Gauss–Seidel meets the target after 19,136 sweeps; its long time mainly reflects one sparse triangular solve per sweep in this implementation.

The Krylov methods need 226–242 iterations. Bound (3.3) guarantees an energy-norm reduction by 10^{-8} within $\frac{1}{2}\sqrt{\kappa}\ln(2\times10^{8}) \approx 912$ CG iterations; CG needs about a quarter of this, because the bound uses only the extreme eigenvalues [2]. Diagonal preconditioning helps little: $\kappa_2(S) = 8029$ is only 12% below $\kappa_2(A)$, since the diagonal of A varies only near the boundary, and PCG saves 16 iterations. Full GMRES needs the fewest iterations but is 35 times slower than CG, the cost of orthogonalizing against all previous basis vectors [1]. At this size, sparse LU factorization and solve together take 2.3 ms, less than any iterative method.

Equal residual targets do not give equal accuracy: the errors of Gauss–Seidel and CG differ by a factor of 180, both within the bound $\kappa_2(A)\,\eta \approx 9.1\times10^{-5}$ from Eq. (3.5). Iterations are algebraic corrections towards one static equilibrium, not physical time steps.

4.3 Deformation and validation

Figure 1c shows the 20×20 solution. The right edge moves between 0.4916 mm (mid-height) and 0.4967 mm (bottom corner), close to $pL/E = 0.5$ mm. Near the loaded end, the top and bottom edges move inwards by 0.077 and 0.079 mm, together matching the free lateral contraction $\nu pL/E = 0.15$ mm. The clamp prevents this contraction at $x = 0$, producing the tapered shape.

For validation, all 42 prescribed components are exactly zero in the reconstructed d. The reactions of Eq. (2.4) sum to -1.000×10^{6} N in x, leaving a force imbalance of $(-4.0, -2.6)\times10^{-9}$ N and a moment imbalance about the origin of 4.5×10^{-9} N m, against an applied moment of 5×10^{5} N m. The five methods that met the target agree with LU to at most 1.7×10^{-7} relative, or 8.3×10^{-8} mm in any component. On the same mesh, the roller-supported patch test of Section 3 (860 unknowns) reproduces the exact field $u_x = px/E$, $u_y = -\nu py/E$ with relative error 2.6×10^{-14}, verifying assembly, loading and reduction. The clamped plate has no closed-form solution, so Section 6 estimates its discretization error by comparing meshes from 5×5 to 80×80 cells, together with the growth of $\kappa_2(A)$ and the iteration counts.

References

[1] Yousef Saad, *Iterative methods for sparse linear systems*, second ed., SIAM, Philadelphia, 2003.

[2] Jonathan Richard Shewchuk, *An introduction to the conjugate gradient method without the agonizing pain*, Technical Report CMU-CS-94-125, School of Computer Science, Carnegie Mellon University, 1994, Edition $1\frac{1}{4}$.